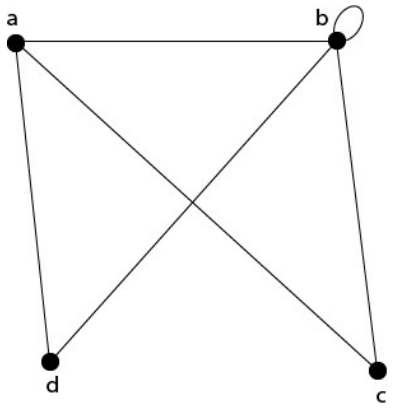
CORE – 7: Discrete Mathematical Structures – Unit-3

Definition of Graph − A graph (denoted as G = (V, E)) consists of a non-empty set of vertices V and a set of edges E.

Example –Let us consider, a Graph G=(V,E) where V={a,b,c,d} and E={{a,b},{a,c},{a,d},{b,b},{b,c},{b,d}}



Degree of a Vertex − The degree of a vertex is the number of edges incident on a vertex v. The self-loop is counted twice. The degree of a vertex is denoted by d(v).

Here, d(a)=3; d(b)=5; d(c) = 2; d(d)=2.

Degree of a Graph − the degree of a graph is the largest vertex degree of that graph. For the above graph the degree of the graph is 5.

Pendant Vertex: A vertex with degree one is called a Pendant Vertex.

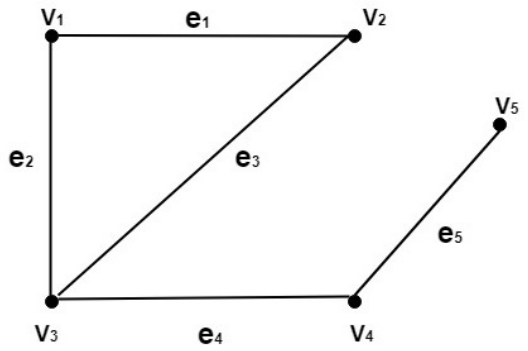
Pendant Edge: The only edge which is an incident with a pendant vertex is called the Pendant Edge.

Odd Vertex: A vertex having degree odd is called an odd vertex.

Even Vertex: A vertex having a degree even is called an even vertex.

Adjacent Vertices: Two vertices are called adjacent if an edge links them. If there is an edge (u, v), then we can say vertex u is adjacent to vertex v, and vertex v is adjacent to vertex u.

Example: Consider the graph as shown in fig:



Determine the following:

1. Pendant Vertices
2. Pendant Edges
3. Odd vertices
4. Even Vertices

Solution:

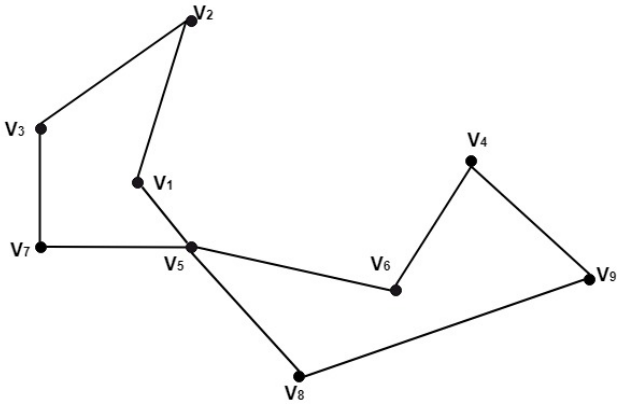
1. The vertex V_5 is the pendant vertex.
2. The edge (V_4,V_5) or e_5 is the pendant edge.
3. The vertices V_3 and V_5 are odd vertices.
4. The vertices V_1, V_2,and V_4 are even vertices.

Cut Set: Consider a graph G = (V, E). A cut set for G is the smallest set of edges such that the removal of the set disconnected the graph whereas the removal of any proper subset of this set left a connected subgraph.

Cut Points or Cut Vertices: Consider a graph G = (V, E). A cut point for a graph G is a vertex v such that G-v has more connected components than G. The subgraph G-v is obtained by deleting the vertex v from graph G and also deleting the entire edges incident on v.

Bridge (Cut Edges): Consider a graph G = (V, E). A bridge for a graph G, is an edge e such that G-e has more connected components than G.

Example-1: Consider the graph shown in the fig. Determine the cut set, cut vertices and cut edges of the graph.

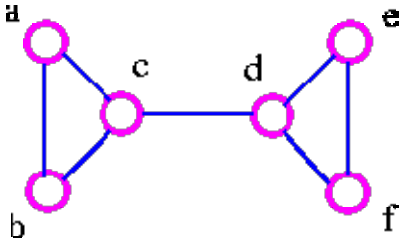


Solution: For this graph, the edge set {(V_1, V5), (V_7, V5)} is a cut set.

V_5 is a cut vertex of the graph.

There is no cut edge in this graph.

Example-2: Consider the graph shown in the fig. Determine the cut set, cut vertices and cut edges of the graph.

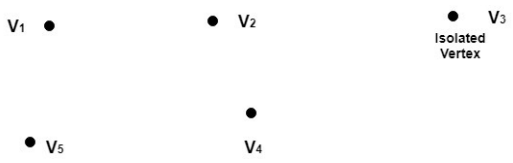


Solution: For this graph, the edge set {(c,d)} is a cut set.
c, d are the cut vertices of the graph.
The edge (c, d) is the cut edge in this graph.

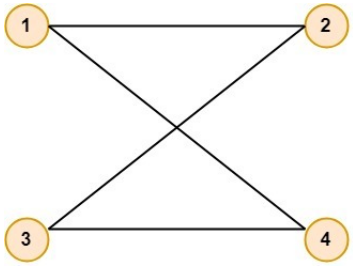
The Handshaking Lemma − in a graph, the sum of all the degrees of all the vertices is equal to twice the number of edges.

Types of Graphs:

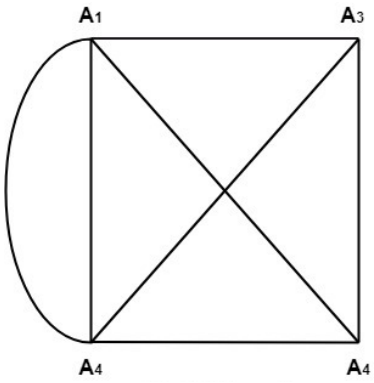
1. Null Graph: A null graph has no edges or in other words a graph having **only isolated vertex.**



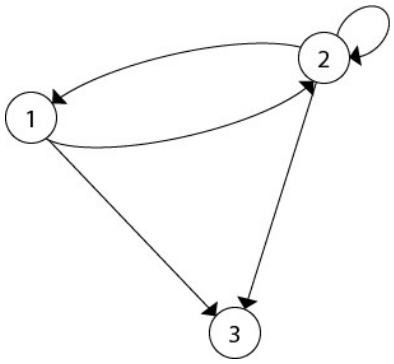
2. Undirected Graphs: An Undirected graph G consists of a set of vertices, V and a set of edge E. The edge set contains the unordered pair of vertices. If (u, v) ∈ E, then we say u and v are connected by an edge where u and v are vertices in the set V.



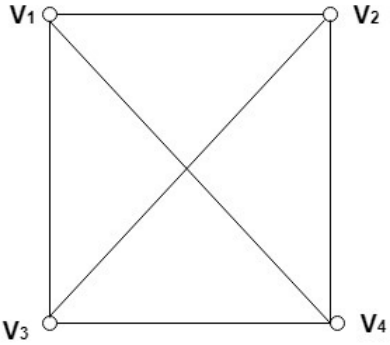
3. Multigraph: If in a graph **multiple edges between the same set of vertices** are allowed, it is known as Multigraph. In other words, it is a graph having at least one loop or multiple edges.



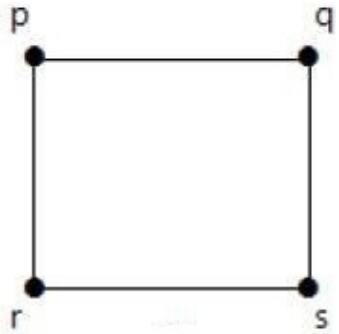
4. Directed Graphs: A directed graph or digraph G is defined as an unordered pair (V, E), where V is the set of points called vertices and E is the set of edges. Each edge in the graph G is assigned a direction and is identified with an ordered pair (u, v), where u is the initial vertex, and v is the end vertex. For a directed graph, the number of edges coming into a vetex is called the **indegree** of the vertex and the number of edges coming out of a vertex is called as **outdegree** of the vertex. Here indegree(3) = 2 and outdegree(3) = 0.



5. Complete Graph: A complete graph G = (V, E) of n vertices is a graph in which each vertex is connected to every other vertex i.e., **an edge exist between every pair of distinct vertices**. It is denoted by $\mathbf{K_n}$. A complete graph with n vertices will have n(n-1)/2 edges.



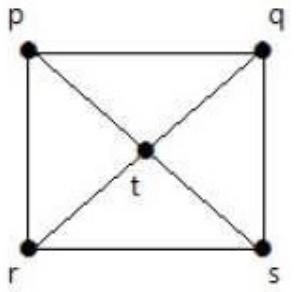
6. Regular Graph: A graph G is said to be regular, if **all its vertices have the same degree**. In a graph, if the degree of each vertex is 'k', then the graph is called a '**k-regular** graph'. In this fig. the graph is **2-regular**.



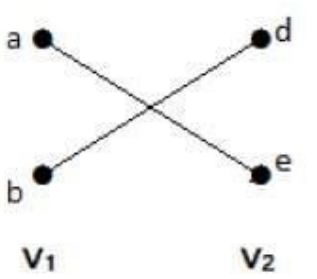
7. Cycle Graph: A simple graph with 'n' vertices (**n >= 3**) and 'n' edges is called a cycle graph if all its edges form a cycle of length 'n' and is denoted as $\mathbf{C_n}$.

Each vertex of a cycle graph has degree 2. Hence it is also a **2-regular** graph. A graph with no cycles is called an **acyclic graph.**

8. Wheel Graph: A wheel graph is obtained from a cycle graph C_n by adding a new vertex. That new vertex is called a **Hub** which is connected to all the vertices of C_n. A wheel graph is denoted as $\mathbf{W_n}$ where **n is the number of edges coming out of the Hub**. Here is an example of $\mathbf{W_4}$ in this fig.

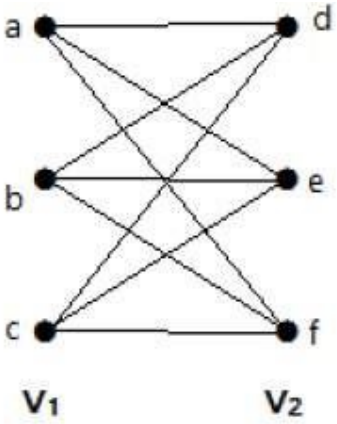


9. Bipartite Graph: A simple graph G = (V, E) with vertex partition V = {V1, V2} is called a bipartite graph if **every edge of E joins a vertex in V1 to a vertex in V2.** In general, a Bipertite graph has two sets of vertices, let us say, V1 and V2, and if an edge is drawn, it should connect any vertex in set V1 to any vertex in set V2 (no edge in G connects either two vertices in V1 or two vertices in V2.



10. Complete Bipartite Graph: A bipartite graph G = (V, E) with partition V = {V1, V2} is said to be a complete bipartite graph if **every vertex in V1 is connected to every vertex of V2** and is denoted by $\mathbf{K_{m,\ n}}$ where m is number of vertices in V_1 and n is number of vertices in V_2.

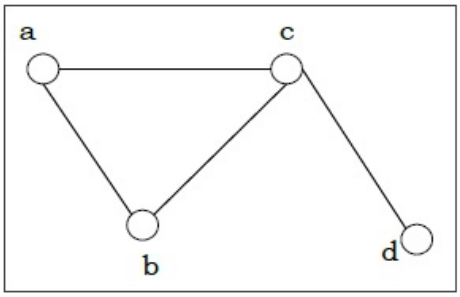
Here is an example of $K_{3,\ 3}$



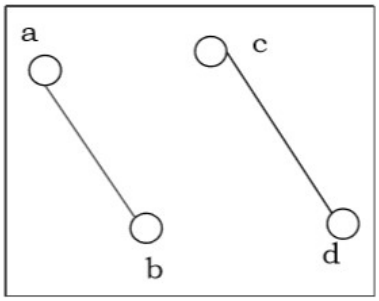
11. Star Graph: A complete bipartite graph of the form $\mathbf{K_{1,\ n-1}}$ is a star graph with n-vertices. A star graph is a complete bipartite graph if a single vertex belongs to one set and all the remaining vertices belong to the other set. Here is an example of $\mathbf{K_{1,\ 4}}$



12. Connected Graph: A graph is called connected **if there is a path from any vertex u to v.**

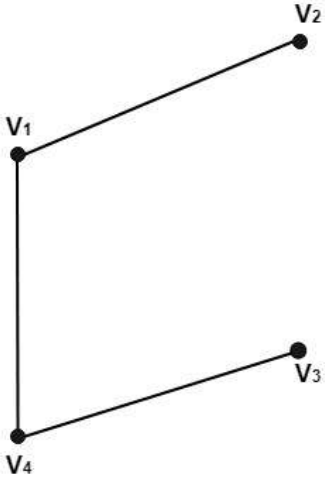


13. Disconnected Graph: A graph is called disconnected **if there is no path between any two of its vertices.**



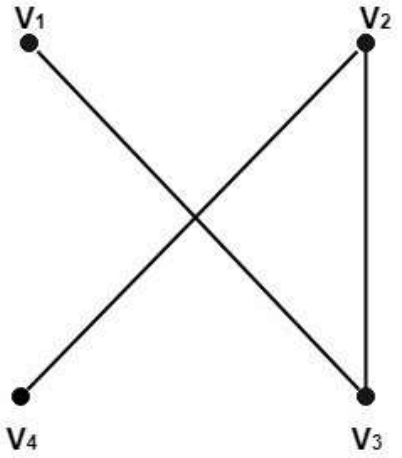
14. Connected Component: A subgraph of graph G is called the connected component of G, if it is not contained in any bigger subgraph of G, which is connected. It is defined by listing its vertices.

Example: In the above disconnected graph the connected components are {a,b} and {c,d}.



(Graph G)

15. Complementary Graph: Complement or inverse of a graph G is a graph H on the same vertices such that two distinct vertices of H are adjacent if and only if they are not adjacent in G.



(Complement of Graph G)

16. Labeled Graphs: A graph G = (V, E) is called a labeled graph if its edges are labeled with some name. So, we can write these labels in place of an ordered pair in its edges set.

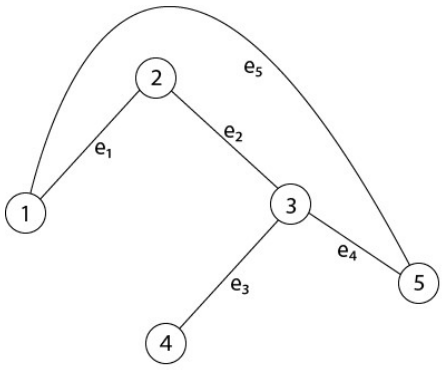


Fig: Undirected Labeled Graph

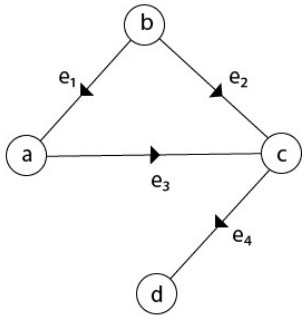
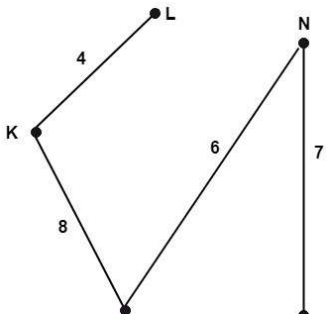


Fig: Directed Labeled Graph

17. Weighted Graphs: A graph G=(V, E) is called a weighted graph if each edge of graph G is assigned a positive number w called the weight of the edge e.



Representation of Graphs: There are two principal ways to represent a graph.

1. Adjacency matrix representation
2. Incidence matrix representation

(a) Representation of Undirected Graph:

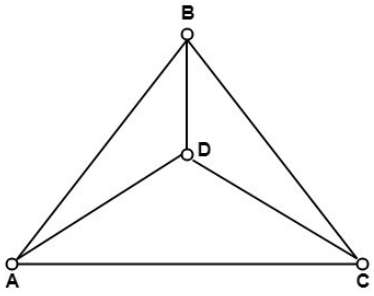
1. Adjacency Matrix Representation: If an Undirected Graph G consists of n vertices then the adjacency matrix of the graph is an n x n matrix A = $[a_{ij}]$ and defined by

$$a_{ij} = \begin{cases} 1, \text{ if } (v_i,v_j) \text{ is an edge i.e. } v_i \text{ is adjacent to } v_j \\ 0, \text{ if there is no edge between } v_i \text{ and } v_j \end{cases}$$

If there exists an edge between vertex v_i and v_j, where i is a row and j is a column then the value of a_{ij}=1.

If there is no edge between vertex v_i and v_j, then value of a_{ij}=0.

Example: Find the adjacency matrix M_A of graph G shown in Fig:



Solution: Since graph G consist of four vertices. Therefore, the adjacency matrix wills a 4 x 4 matrix. The adjacency matrix is as follows in fig:

$M_A =$

	A	B	C	D
A	0	1	1	1
B	1	0	1	1
C	1	1	0	1
D	1	1	1	0

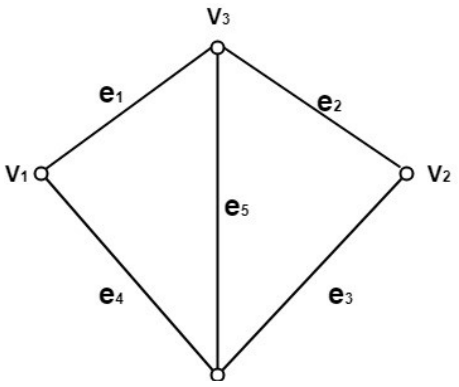
2. Incidence Matrix Representation: If an Undirected Graph G consists of n vertices and m edges, then the incidence matrix is an n x m matrix C = $[c_{ij}]$ and is defined by

$$c_{ij} = \begin{cases} 1, \text{ if the vertex } v_i \text{ incident by } e_j \\ 0, \text{ otherwise} \end{cases}$$

There is a row for every vertex and a column for every edge in the incident matrix.

The number of ones in an incidence matrix of the undirected graph (without loops) is equal to the sum of the degrees of all the vertices in a graph. It is also equal to twice the number of edges.

Example: Consider the undirected graph G as shown in fig. Find its incidence matrix M_I.



Solution: The undirected graph consists of four vertices and five

$M_I =$

	e_1	e_2	e_3	e_4	e_5
V_1	1	0	0	1	0
V_2	0	1	1	0	0
V_3	1	1	0	0	1
V_4	0	0	1	1	1

Therefore, the incidence matrix is an 4 x 5 matrix.

(b)Representation of Directed Graph:

1. Adjacency Matrix Representation: If a directed graph G consists of n vertices then the adjacency matrix of a graph is an n x n matrix A = $[a_{ij}]$ and defined by

$$a_{ij} = \begin{cases} 1, \text{ if } (v_i,v_j) \text{ is an edge i.e. } v_i \text{ is initial vertex and } v_j \text{ is final vertex} \\ 0, \text{ if there is no edge between } v_i \text{ and } v_j \end{cases}$$

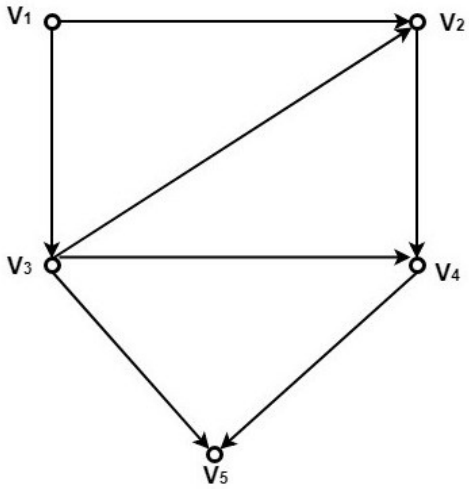
If there exists an edge between vertex V_i and V_j, with V_i as initial vertex and V_j as a final vertex, then the value of a_{ij}=1.

If there is no edge between vertex V_i and V_j, then the value of a_{ij}=0.

The number of ones in the adjacency matrix of a directed graph is equal to the number of edges.

Example: Consider the directed graph shown in fig. Determine its adjacency matrix M_A.

Solution: Since the directed graph G consists of five vertices. Therefore, the adjacency matrix will be a 5 x 5 matrix as shown in th fig.



$M_A =$

	V_1	V_2	V_3	V_4	V_5
V_1	0	1	1	0	0
V_2	0	0	0	1	0
V_3	0	1	0	1	1
V_4	0	0	0	0	1
V_5	0	0	0	0	0

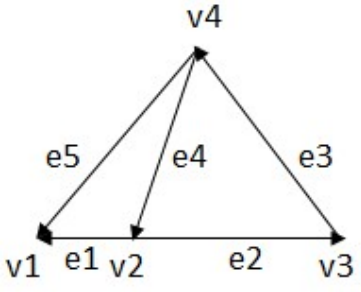
2. Incidence Matrix Representation: If a directed graph G consists of n vertices and m edges, then the incidence matrix is an n x m matrix C = $[c_{ij}]$ and defined by

$$a_{ij} = \begin{cases} 1, v_i \text{ is the initial vertex of edge } e_i \\ -1, v_i \text{ is the final vertex of edge } e_i \\ 0, \text{ if } v_i \text{ is not incident on } e_i \end{cases}$$

The number of ones in an incidence matrix is equal to the number of edges in the graph.

Example: Consider the directed graph G as shown in fig. Find its incidence matrix M_I.

Solution: The directed graph consists of four vertices and five edges.



$M_I =$

	e_1	e_2	e_3	e_4	e_5
V_1	-1	0	0	0	-1
V_2	1	1	0	-1	0
V_3	0	-1	1	0	0
V_4	0	0	-1	1	1

Therefore, the incidence matrix is a 4 x 5 matrix as shown in the fig.

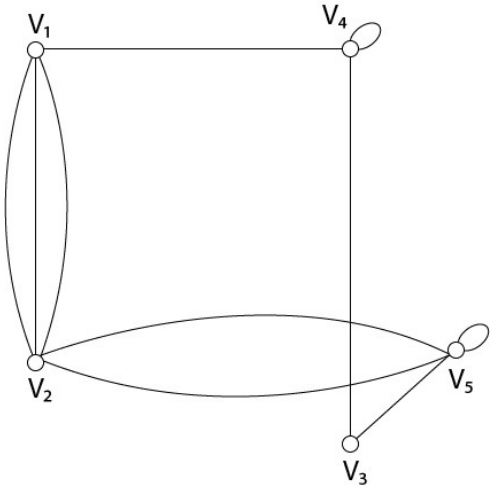
(c) Representation of Multigraph (Represented only by adjacency matrix representation)

(i) Adjacency matrix representation of multigraph: If a multigraph G consists of vertices, then the adjacency matrix of graph is an n x n matrix A = $[a_{ij}]$ and is defined by

a_{ij} = N, where N is the number of edges between V_i and V_j

a_{ij} = 0 If there is no edge between v_i and v_j.

Example:
Consider the multigraph shown in Fig, Determine its adjacency matrix.



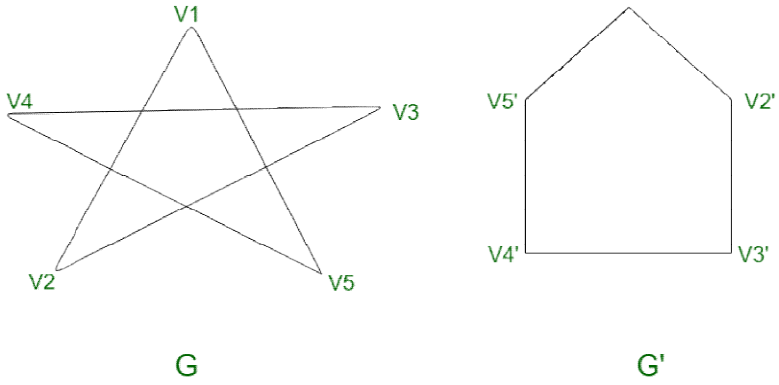
	V_1	V_2	V_3	V_4	V_5
V_1	0	3	0	1	0
V_2	3	0	0	0	2
V_3	0	0	0	1	1
V_4	1	0	1	1	0
V_5	0	2	1	0	1

Solution:
Since the multigraph consists of five vertices, therefore the adjacency matrix will be a 5 x 5 matrix.

The adjacency matrix of the multigraph is as shown in the above figure.

Isomorphism:

Graphs G(V, E) and G* (V*,E*) are said to be isomorphic if there exists one to one correspondence i.e. f:V→V* such that {u, v} is an edge of G if and only if {f(u), f(v)} is an edge of G* i.e. if u,v are adjacent then f(u) and f(v) are also adjacent. So, f must be a bijective function.



(In the above figure graph G and graph G' are iosmerphic graphs.)

Euler paths and circuits:

- An Euler path is a path that uses every edge of a graph **exactly once**.
- An Euler circuit is a circuit that uses every edge of a graph exactly once.
- An Euler path starts and ends at different vertices.
- An Euler circuit starts and ends at the same vertex.

Theorem – "A connected multigraph (and simple graph) with at least two vertices has a Euler circuit if and only if **each** of its vertices has an **even degree**."

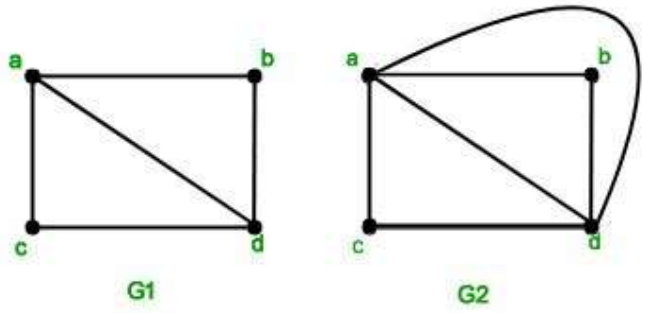
Proof of the above statement is that every time a circuit passes through a vertex, it adds twice to its degree. Since it is a circuit, it starts and ends at the same vertex

which makes it contribute one degree when the circuit starts and one when it ends. In this way, every vertex has an even degree.

Theorem – "A connected multigraph (and simple graph) has an Euler path but not an Euler circuit if and only if it has **exactly two vertices** of odd degree."

Proof: Since a path may start and end at different vertices, the vertices where the path starts and ends are allowed to have odd degrees.

Example – Which graphs shown in the fig. have an Euler path or Euler circuit?



Solution – G_1 has two vertices of odd degree a and d and the rest of them have even degree. So this graph has an Euler path but not an Euler circuit.
The path starts and ends at the vertices of odd degree. The path is- a,c,d,a,b,d.G_2 has four vertices all of even degree, so it has a Euler circuit. The circuit is- a,d,b,a,c,d,a.

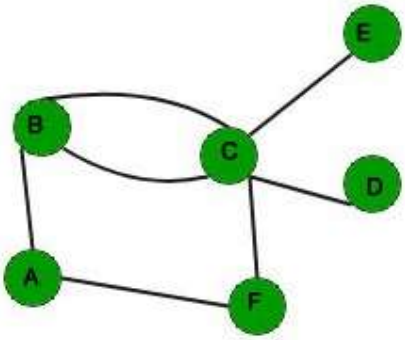
Hamiltonian paths and circuits:

Hamilonian Path – A simple path in a graph that passes through **every vertex exactly once** is called a Hamiltonian path.

Hamilonian Circuit – A simple circuit in a graph that passes through every vertex exactly once is called a Hamiltonian circuit.

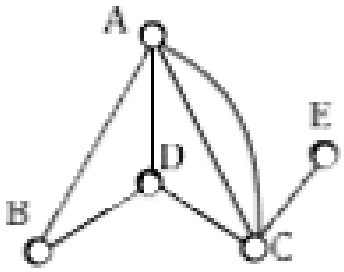
Unlike Euler paths and circuits, there are no simple necessary and sufficient criteria to determine if there are any Hamiltonian paths or circuits in a graph. But there are certain criteria which rule out the existence of a Hamiltonian circuit in a graph, such as- if there is a vertex of degree one in a graph then it is impossible for it to have a Hamiltonian circuit. There are certain theorems which give sufficient but not necessary conditions for the existence of Hamiltonian graphs.

Example-1: Does the following graph have a Hamiltonian Circuit?



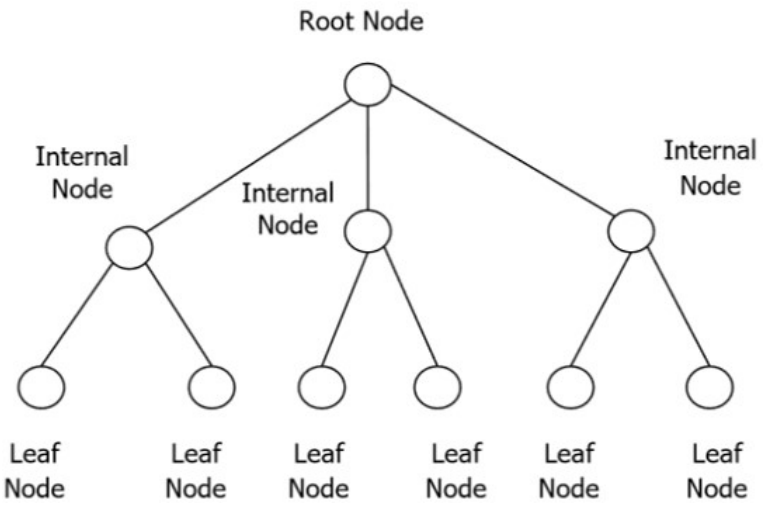
Solution- No the above graph does not have a Hamiltonian circuit as there are two vertices with degree one in the graph.

Example-2: Does a Hamiltonian path or circuit exist on the graph below?



Solution- We can see that once we travel to vertex E there is no way to leave without returning to C, so there is no possibility of a Hamiltonian circuit. If we start at vertex E we can find several Hamiltonian paths, such as ECDAB and ECABD.

Trees: a **tree** is an **undirected, connected and acyclic graph**. In other words, a connected graph that does not contain any cycle is called a tree.



Binary Trees: If the outdegree of every node is less than or equal to 2, in a directed tree than the tree is called a binary tree.

Basic Terminology:

Root: A binary tree has a unique node called the root of the tree.

Left Child: The node to the left of the root is called its left child.

Right Child: The node to the right of the root is called its right child.

Parent: A node having a left child or right child or both are called the parent of the nodes.

Siblings: Two nodes having the same parent are called siblings.

Leaf: A node with no children is called a leaf.

Descendant: A node is called descendant of another node if it is the child of the node or child of some other descendant of that node. All the nodes in the tree are descendants of the root.

Left Subtree: The subtree whose root is the left child of some node is called the left subtree of that node.

Right Subtree: The subtree whose root is the right child of some node is called the right subtree of that node.

Level of a Node: The level of a node is its distance from the root. The level of root is defined as zero. The level of all other nodes is one more than its parent node. The maximum number of nodes at any level N is 2^N.

Depth or Height of a tree: The depth or height of a tree is defined as the maximum number of nodes in a branch of a tree. This is more than the maximum level of the tree, i.e., the depth of root is one. The maximum number of nodes in a binary tree of depth d is 2^{d+1} -1.

External Nodes: The nodes which have no children are called external nodes or terminal nodes.

Internal Nodes: The nodes which have one or more children are called internal nodes or non-terminal nodes.

Traversing Binary Trees:

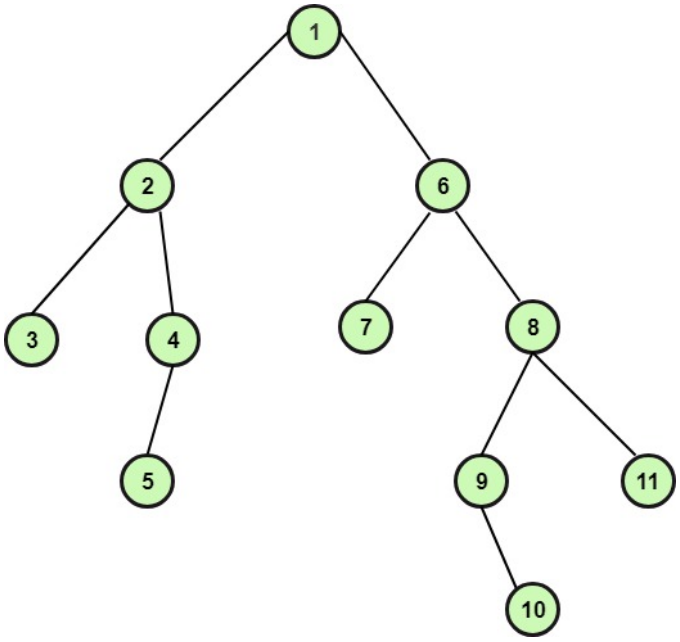
Traversing means visiting each node of the tree exactly once. There are three traversal methods of binary trees.

1. Preorder Traversal
2. Postorder Traversal
3. Inorder Traversal

1. Preorder Traversal: The preorder traversal of a binary tree is a recursive process. The preorder traversal of a tree is

- Visit the root of the tree.
- Traverse the left subtree in preorder.
- Traverse the right subtree in preorder.

Example: Determine the preorder, postorder and inorder traversal of the following binary tree.



2. Postorder Traversal: The postorder traversal of a binary tree is a recursive process. The postorder traversal of a tree is

- Traverse the left subtree in postorder.
- Traverse the right subtree in postorder.
- Visit the root of the tree.

3. Inorder Traversal: The inorder traversal of a binary tree is a recursive process. The inorder traversal of a tree is

- Traverse the left subtree in inorder.
- Visit the root of the tree.
- Traverse the right subtree in inorder.

Solution:

Preorder	1	2	3	4	5	6	7	8	9	10	11
Postorder	3	5	4	2	7	10	9	11	8	6	1
Inorder	3	2	5	4	1	7	6	9	10	8	11

Spanning Tree: A spanning tree is a subset of Connected Graph G, which has all the vertices covered that does not have cycles. By this definition, we can draw a conclusion that every connected and undirected Graph G has at least one spanning tree.

Minimum Spanning Tree: Suppose G is a connected weight graph i.e., each edge of G is assigned a non-negative number called the weight of edge, then any spanning tree T of G is assigned a total weight obtained by adding weights of the all the edges in T. A minimum spanning tree of G is a tree whose total weight is as small as possible.

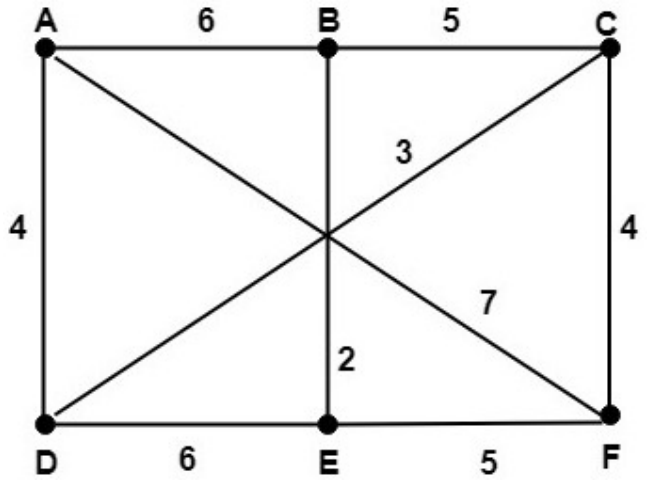
Kruskal's Algorithm to find a minimum spanning tree:

1. Input the given connected weighted graph G with n vertices whose minimum spanning tree T, we want to find.
2. Order all the edges of the graph G according to increasing weights.
3. Initialize T with all the vertices of G.
4. Go on adding edges of the graphs G in T which does not form a cycle from the ordered list in step-2 until n-1 edges are added, where n is number of vertices in the graph G.

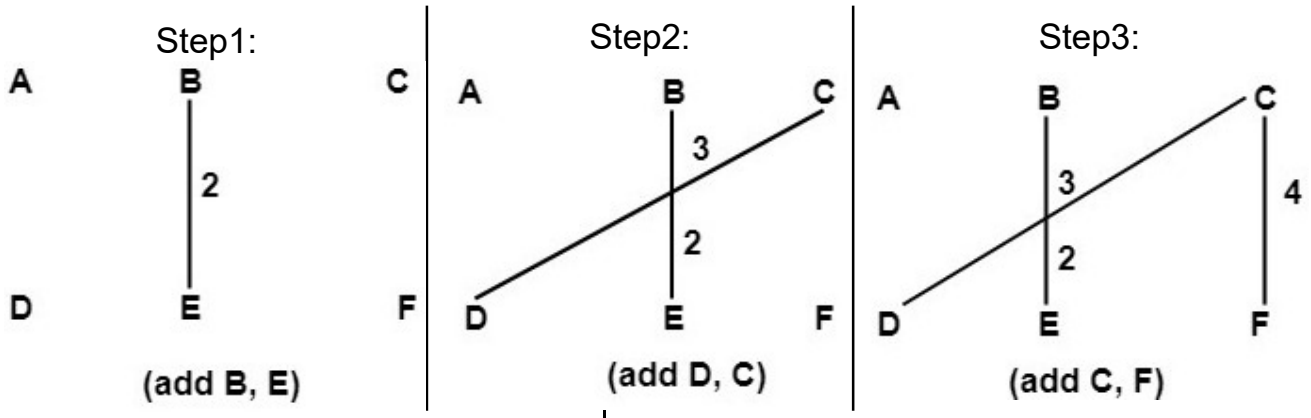
Example: Determine the minimum spanning tree of the weighted graph shown in the following figure.

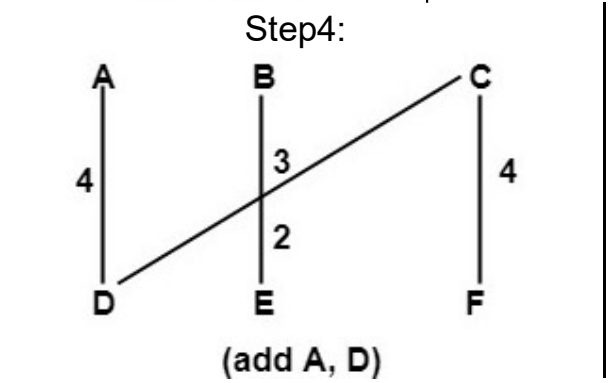
Solution:

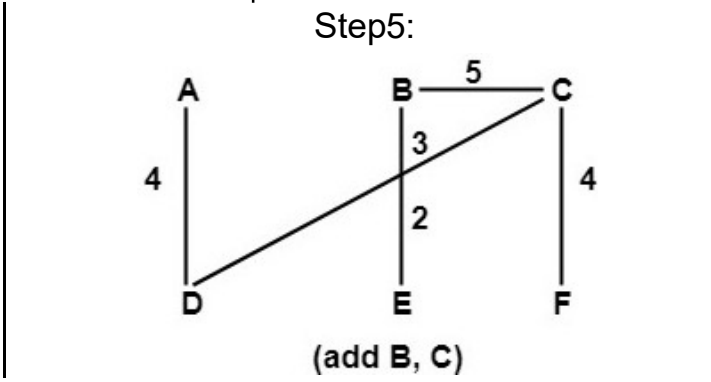
Edges	Weights	Added or Not
(B, E)	2	Added
(C, D)	3	Added
(A, D)	4	Added
(C, F)	4	Added
(B, C)	5	Added
(E, F)	5	Not added
(A, B)	6	Not added
(D, E)	6	Not added
(A, F)	7	Not added



Using kruskal's algorithm arrange all the edges of the weighted graph in increasing order and initialize spanning tree T with all the six vertices of G. Now start adding the edges of G in T which do not form a cycle and having minimum weights until five edges are not added as there are six vertices.

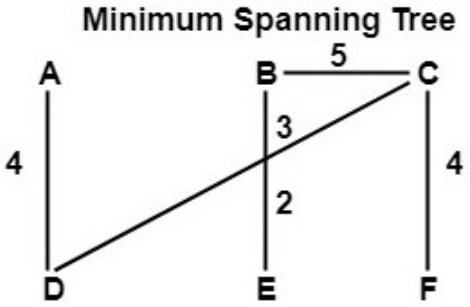






Step6: Edge (A, B), (D, E) and (E, F) are discarded because they will form the cycle in a graph.

So, the minimum spanning tree form in step 5 is output, and the total cost is 18.



<u>Graph Traversal Algorithm</u>: Traversing the graph means examining all the nodes or vertices of the graph. There are two standard graph traversal methods:

- Breadth First Search
- Depth First Search

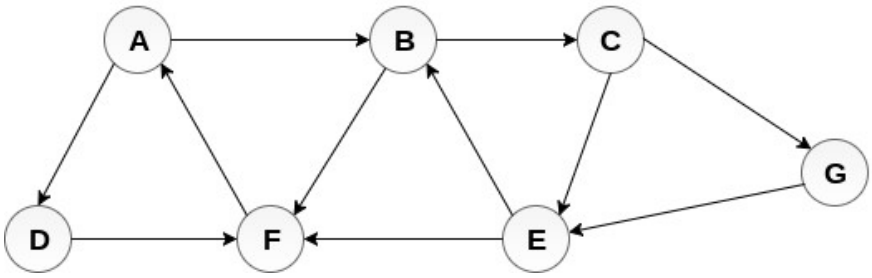
<u>Breadth First Search (BFS) Algorithm</u>: Breadth first search is a graph traversal algorithm that starts traversing the graph from any node and explores all the neighbouring nodes until the goal is reached. In each iteration, it selects the nearest unexplored node. The algorithm follows the same process for each of the nearest node until it finds the goal. The algorithm ensures that each node is visited exactly once and no node is visited twice. It uses the data structure **QUEUE**.

Algorithm

- **Step 1:** SET STATUS = 1 (ready state) for each node in G
- **Step 2:** Enqueue the starting node A and set its STATUS = 2 (waiting state)
- **Step 3: Repeat** Steps 4 and 5 **until QUEUE is empty or the goal is reached**

- **Step 4:** Dequeue a node N. Process it and set its STATUS = 3 (processed state).
- **Step 5:** Enqueue all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)
  [END OF LOOP]
- **Step 6:** EXIT

Example: Consider the graph G shown in the following fig. calculate the minimum path p from node A to node E by using BFS algorithm. Given that each edge has a length of 1.



Adjacency Lists

A : B, D
B : C, F
C : E, G
G : E
E : B, F
F : A
D : F

Solution:
The algorithm uses two queues, namely **QUEUE1** and **QUEUE2**. **QUEUE1** holds all the nodes that are to be processed while **QUEUE2** holds all the nodes that are processed and deleted from **QUEUE1**.

Let's start examining the graph from Node A.

1. Add A to QUEUE1 and NULL to QUEUE2.
   So, QUEUE1 = {A} QUEUE2 = {NULL}
2. Delete the Node A from QUEUE1 and insert all its neighbours. Insert Node A into QUEUE2
   So, QUEUE1 = {B, D} QUEUE2 = {A}
3. Delete the node B from QUEUE1 and insert all its neighbours. Insert node B into QUEUE2.
   So, QUEUE1 = {D, C, F} QUEUE2 = {A, B}
4. Delete the node D from QUEUE1 and insert all its neighbours. Since F is the only neighbour of it which has been inserted, we will not insert it again. Insert node D into QUEUE2.
   So, QUEUE1 = {C, F} QUEUE2 = { A, B, D}
5. Delete the node C from QUEUE1 and insert all its neighbours. Add node C to QUEUE2.
   So, QUEUE1 = {F, E, G} QUEUE2 = {A, B, D, C}

6. Remove F from QUEUE1 and add all its neighbours. Since all of its neighbours have already been added, we will not add them again. Add node F to QUEUE2.
   So, QUEUE1 = {E, G} QUEUE2 = {A, B, D, C, F}
7. Remove E from QUEUE1, all of E's neighbours has already been added to QUEUE1 therefore we will not add them again. All the nodes are visited and the target node i.e. E is encountered into QUEUE2.
   So, QUEUE1 = {G} QUEUE2 = {A, B, D, C, F, E}

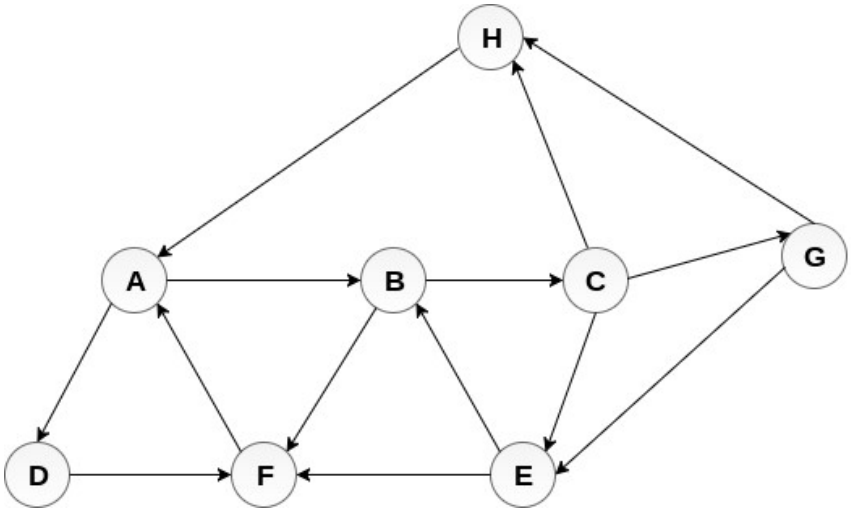
Now, backtrack from E to A, using the nodes available in QUEUE2.
The minimum path will be **A → B → C → E**.

Depth First Search (DFS) Algorithm: Depth first search (DFS) algorithm starts with the initial node of the graph G, and then goes to deeper and deeper until we find the goal node or the node which has no children. The algorithm, then backtracks from the dead end towards the most recent node that is yet to be completely unexplored. It uses the data structure **STACK.**

Algorithm

- **Step 1:** SET STATUS = 1 (ready state) for each node in G
- **Step 2:** Push the starting node A on the stack and set its STATUS = 2 (waiting state)
- **Step 3: Repeat** Steps 4 and 5 **until STACK is empty or the goal is reached**
- **Step 4:** Pop the top node N. Process it and set its STATUS = 3 (processed state)
- **Step 5:** Push on the stack all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)
  [END OF LOOP]
- **Step 6:** EXIT

Example: Consider the graph G along with its adjacency list, given in the figure below. Calculate the order to print all the nodes of the graph starting from node H, by using depth first search (DFS) algorithm.



Adjacency Lists

A : B, D
B : C, F
C : E, G, H
G : E, H
E : B, F
F : A
D : F
H : A

Solution:

1. Push H onto the stack, STACK : H
2. POP the top e**lem**ent of the stack i.e. H, print it and push all the neighbours of H onto the stack that are is ready state, **Print H** STACK: A
3. Pop the top element of the stack i.e. A, print it and push all the neighbours of A onto the stack that are in ready state, **Print A** Stack: B, D
4. Pop the top element of the stack i.e. D, print it and push all the neighbours of D onto the stack that are in ready state, **Print D** Stack: B, F
5. Pop the top element of the stack i.e. F, print it and push all the neighbours of F onto the stack that are in ready state, **Print F** Stack: B
6. Pop the top of the stack i.e. B and push all the neighbours,
   Print B Stack: C
7. Pop the top of the stack i.e. C and push all the neighbours,
   Print C Stack: E, G
8. Pop the top of the stack i.e. G and push all its neighbours,
   Print G Stack: E
9. Pop the top of the stack i.e. E and push all its neighbours,
   Print E Stack: ∅

Hence, the stack now becomes empty and all the nodes of the graph have been traversed. So, the printing sequence of the graph will be:
H → A → D → F → B → C → G → E